
Unit – 2

I. Syntax Analysis:

Overview of Context Free Grammar. Derivations and Parse trees, Ambiguity, Left Recursion, Left factoring.

II. Top down Parsing:

Recursive descent parsing and Predictive parsers.

III. YACC:

YACC parser generator

1. SYNTAX ANALYSIS / PARSING

1.1 The Role of the Syntax Analyzer (Parser)

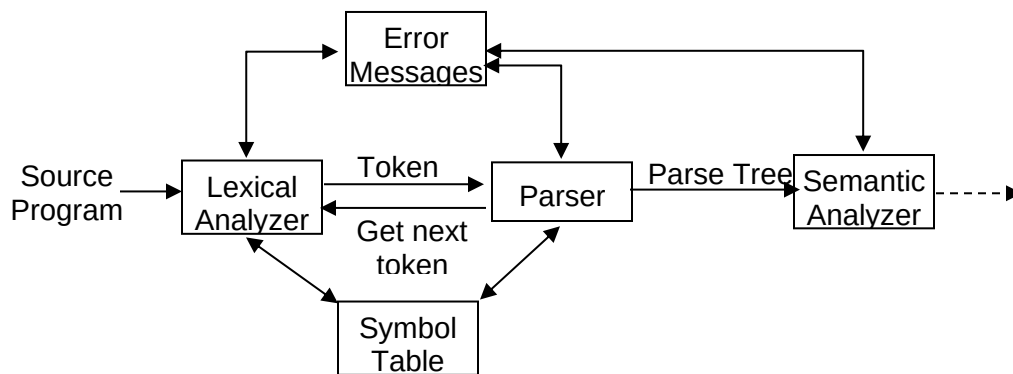


Fig 3.1: Location of the Parser

- The Parser obtains a string of tokens from the lexical analyzer and verifies that the string of tokens can be generated by the grammar for the source language.
- The parser constructs the parse tree and passes it to the rest of the compiler phases for further processing.
- It reports the presence of syntax error and also recovers from commonly occurring errors to continue processing the remainder of the program.
- There are two general types of parsers for grammars
 - Top down parsing – which build the parse tree from root to leaves.
 - Bottom up parsing – which build the parse tree from leaf and work up the root.

Parsing

- Parsing is the process of determining a string of tokens that can be generated by a grammar” or not. For any CFG, there is a parser that parses a string of n tokens.
- Programming language parsers almost always make a single left-to-right scan over the input, looking ahead one token at a time.

1.2 Context Free Grammars

- Grammars are used to systematically describe the syntax of programming language constructs like expressions and statements.
- **Significance of Grammars in any Programming Language:**

Grammars offer significant benefits for both language designers and Compiler writers.

 - A Grammar gives a precise, easy to understand, syntactic specification of a programming language.
 - The Parser (syntax analyzers) built from a grammar that determines the syntactic structure of the source program.
 - The structure imparted to a language by a properly augmented grammar is useful for
 - translating source program into object program
 - detecting errors
 - A Grammar allows a language to be evolved or developed iteratively, by adding new constructs to perform new tasks.
- A **Context-Free Grammar** (CFG) consists of Terminals, Non-terminals, a Start symbol, and Productions.
 - **Terminals** are the basic symbols from which strings are formed. The term the terminals are the first components of the tokens output by the lexical analyzer.
 - **Non-Terminals** are syntactic variables that denote sets of strings. The sets of strings denoted by non-terminals help define the language generated by the grammar. Non-terminals impose a hierarchical structure on the language that is key to syntax analysis and translation.
 - In a grammar, one non-terminal is distinguished as the **Start symbol**, and the set of strings it denotes is the language generated by the grammar. Conventionally, the productions for the start symbol are listed first.
 - The **productions** of a grammar specify the manner in which the terminals and non-terminals can be combined to form strings.
 - Each *production* consists of:
 - A non-terminal called the *head* or *left side* of the production; this production defines some of the strings denoted by the head.
 - A *body* or *right side* consisting of zero or more terminals and non-terminals.
 - The components of the body describe one way in which strings of the non-terminal at the head can be constructed.

Notational Conventions:

- **Terminals:** Lowercase letters early in the alphabet, such as a, b , Operators, Punctuation symbols such as parentheses and the digits.
- **Non-terminals:** Uppercase letters early in the alphabet, such as A, B, C . Lowercase, italic names such as *expr* or *stmt*.
- **Grammar Symbol** that is, either non-terminals or terminals, Uppercase letters late in the alphabet, such as X, Y, Z , represent single *grammar symbol*. Lowercase Greek letters α, β, γ represent (possibly empty) strings of grammar symbols.
- A generic **production** can be written as $A \rightarrow a$, where A is the head and ' a ' the body. A set of productions $A \rightarrow a_1, A \rightarrow a_2, \dots, A \rightarrow a_k$ with a common head A (call them *A-productions*), may be written $A \rightarrow a_1 \mid a_2 \mid \dots \mid a_k$.

1.3 Verifying the language generated by the Grammar

- A Grammar G generates a language L has two parts: Every string generated by G is in L and conversely every string in L can indeed be generated by G .
- The *language generated by* a grammar is its set of sentences. Thus, a string of terminals w is in $L(G)$, the language generated by G , if and only if w is a sentence of G .
- **Sentential form** may contain both terminals and non-terminals, and may be empty.
- A **sentence or string of G** is a sentential form with no non-terminals.
- If two grammars generate the same language, the grammars are said to be *equivalent*.
- The techniques that are used to verify the given string is generated by G are,
 - Derivation
 - Reduction

(i) Derivation

- The derivation corresponds to the top down construction of a parse tree.
- Derivation beginning with the Start symbol of the grammar G .
- Each rewriting step replaces a non-terminal by the body of (rhs) one of its productions.
- Derivation can be Left-most and Right-most derivation in which the non-terminal to be replaced at each step is chosen.
 - In **Left-most Derivation (LMD)**, the leftmost non-terminal in each sentential form is always chosen.
 - In **Rightmost derivations (RMD)**, the rightmost non-terminal is always chosen.

Example:

Consider the Grammar,

$S \rightarrow (L) \mid a$

$L \rightarrow L, S \mid S$

and the input string $(a,(a,a))$

Left-most Derivation		Right-most Derivation	
S	Expanded by $S \rightarrow (L)$	S	Expanded by $S \rightarrow (L)$
(L)	Expanded by $L \rightarrow L, S$	(L)	Expanded by $L \rightarrow L, S$
(L,S)	Expanded by $L \rightarrow S$	(L,S)	Expanded by $S \rightarrow (L)$
(S,S)	Expanded by $S \rightarrow a$	(L,(L))	Expanded by $L \rightarrow L, S$
(a,S)	Expanded by $S \rightarrow (L)$	(L,(L,S))	Expanded by $S \rightarrow a$
(a,(L))	Expanded by $L \rightarrow L, S$	(L,(L,a))	Expanded by $L \rightarrow S$
(a,(L,S))	Expanded by $L \rightarrow S$	(L,(S,a))	Expanded by $S \rightarrow a$
(a,(S,S))	Expanded by $S \rightarrow a$	(L,(a,a))	Expanded by $L \rightarrow S$
(a,(a,S))	Expanded by $S \rightarrow a$	(S,(a,a))	Expanded by $S \rightarrow a$
(a,(a,a))	Verified	(a,(a,a))	Verified.

(ii) Reduction

- Reduction is the process of "reducing" a string w to the start symbol of the grammar.
- At each *reduction* step, a specific substring matching the body (RHS) of a production is replaced by the non-terminal at the head of that production.
- The key decisions during bottom-up parsing are about when to reduce and- about what production to apply, as the parse proceeds. Formally a "handle" is a substring that matches the body of a production, and whose reduction represents one step along the reverse of a rightmost derivation.

- Example Consider the Grammar,
 $S \rightarrow (L) \mid a$
 $L \rightarrow L, S \mid S$
 and the input string $(a,(a,a))$

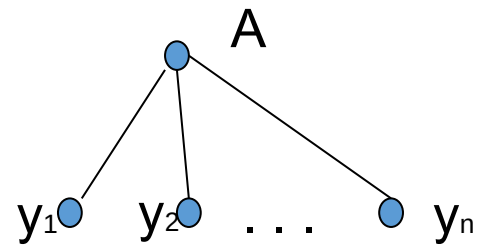
Sentential Form	String	Handle	Reduction Steps (handle pruning)
(	(a,(a,a))		
(a	,(a,a))	a	Reduce by $S \rightarrow a$
(S	,(a,a))	S	Reduce by $L \rightarrow S$
(L,	(a,a))		
(L, (	a,a))		
(L,(a	,a))	a	Reduce by $S \rightarrow a$
(L,(S	,a))	S	Reduce by $L \rightarrow S$
(L,(L,	a))		
(L,(L,a	))	a	Reduce by $S \rightarrow a$
(L,(L,S	))	L,S	Reduce by $L \rightarrow L,S$
(L,(L)	)	(L)	Reduce by $S \rightarrow (L)$
(L,S	)	L,S	Reduce by $L \rightarrow L,S$
(L)	End	(L)	Reduce by $S \rightarrow (L)$
S			Start Symbol of the Grammar.

1.4 Parse Tree

- A parse tree is a graphical representation for a derivation that filters out the choice regarding replacement order.
- It pictorially shows how the start symbol of a grammar derives a string in a language.

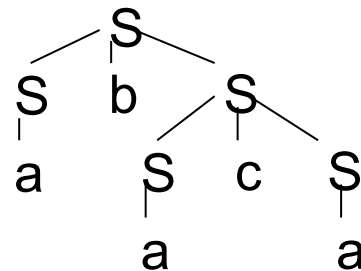
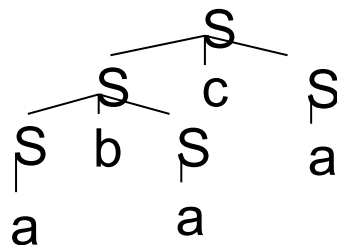
Properties of Parse Tree

1. The Root is labeled by the start symbol of G.
2. Each leaf is labeled by a Token or by ϵ .
3. Each interior node is labeled by a non-terminal
4. If there is a production $A \rightarrow Y_1 Y_2 \dots Y_n$ then A is a interior node with the children of that node are X_1, X_2 and X_3 from left-to-right.



Example: Consider the CFG $G: S \rightarrow SbS \mid ScS \mid a$ and the input string “abaca”

The left-most derivation and right-most derivation of the string can be represented using a parse tree



1.5 ELIMINATION OF LEFT RECURSION

- A Grammar is left recursive if it has a non-terminal A, such that there is derivation $A \rightarrow A\alpha$ for some string α . Top Down parsers cannot handle left-recursive grammars, so a transformation is needed to eliminate left recursion.

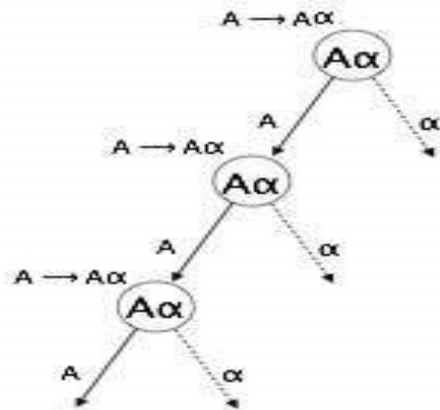
There is a production of the form

$$A \rightarrow A\alpha \mid \beta$$

can be replaced by the non-left recursive pairs,

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' \mid \epsilon, \text{ where } A' \text{ is a new non-terminal.}$$



Example:

$E \rightarrow E + T \mid T$

$E \rightarrow \underline{E + T} \mid \underline{T}$

$A \rightarrow A \alpha \mid \beta$

After eliminating left-recursion,

$E \rightarrow T E'$

$E' \rightarrow +TE' \mid \epsilon$

1.6 ELIMINATION OF LEFT FACTOR

- When the choice between two alternatives A – productions is not clear, a parser may be able to rewrite the productions to defer the decision enough of the input has been seen that can make the right choice.
- For each non-terminal A, find the longest prefix α common to two or more of its alternatives. If there is a nontrivial common prefix replaces all of the A –productions $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \dots \mid \alpha\beta_n \mid \gamma$, where γ represents all alternatives that do not begin with α , then

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \dots \mid \alpha\beta_n \mid \gamma$$

can be rewritten as

$$A \rightarrow \alpha A' \mid \gamma$$

$$A' \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$$

Where A' is a new non-terminal.

Example:

$$S \rightarrow \underline{iEtS} \mid \underline{iEtSeS} \mid a$$

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \gamma$$

$$S \rightarrow \underline{iEtS} \mid \underline{iEtSeS} \mid a$$

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \gamma$$

After eliminating left-factor,

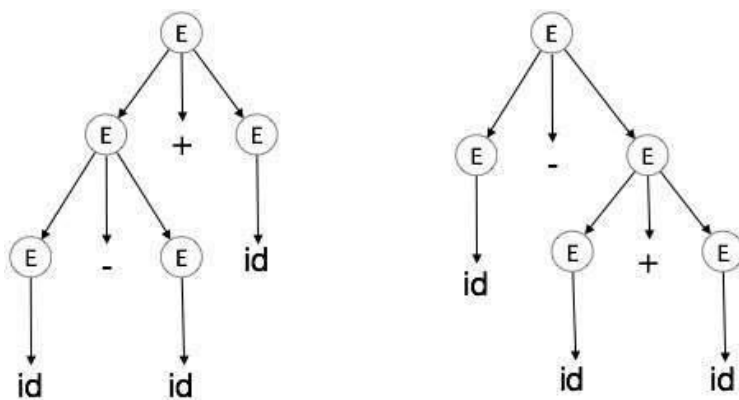
$$S \rightarrow iEtS S' \mid a$$

$$S' \rightarrow eS \mid \epsilon \quad (\text{since } \beta_1 \text{ is } \epsilon)$$

AMBIGUITY:

A grammar G is said to be ambiguous if it has more than one parse tree (left or right derivation) for at least one string.

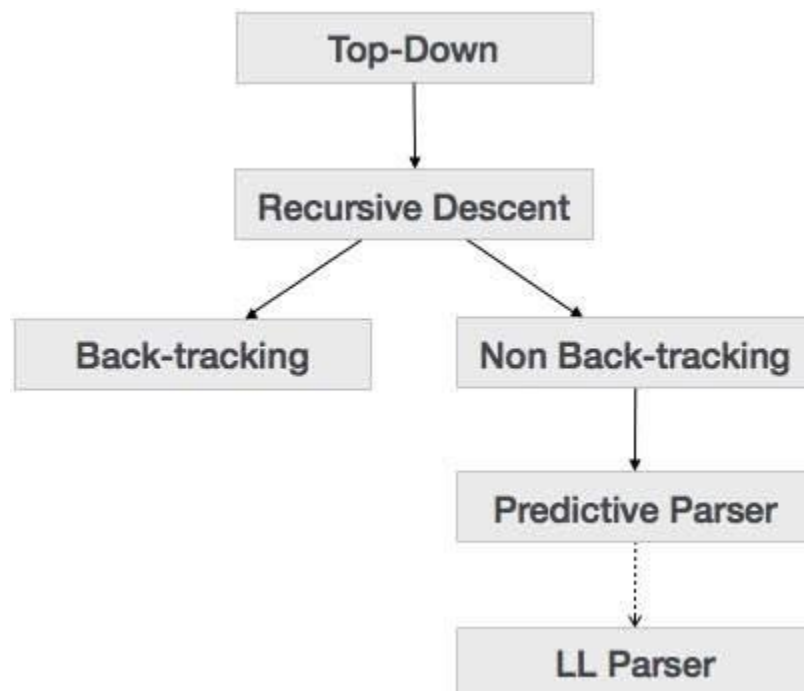
Example: $E \rightarrow E + E; E \rightarrow E - E; E \rightarrow id$



The language generated by an ambiguous grammar is said to be **inherently ambiguous**. Ambiguity in grammar is not good for a compiler construction. No method can detect and remove

ambiguity automatically, but it can be removed by either re-writing the whole grammar without ambiguity, or by setting and following associativity and precedence constraints.

II. TOP DOWN PARSING



2.1 RECURSIVE DESCENT PARSER

Recursive descent is a top-down parsing technique that constructs the parse tree from the top and the input is read from left to right. It uses procedures for every terminal and non-terminal entity. This parsing technique recursively parses the input to make a parse tree, which may or may not require back-tracking. But the grammar associated with it (if not left factored) cannot avoid back-tracking. A form of recursive-descent parsing that does not require any back-tracking is known as **predictive parsing**.

This parsing technique is regarded recursive as it uses context-free grammar which is recursive in nature.

Back-tracking

Top- down parsers start from the root node (start symbol) and match the input string against the production rules to replace them (if matched). To understand this, take the following example of CFG:

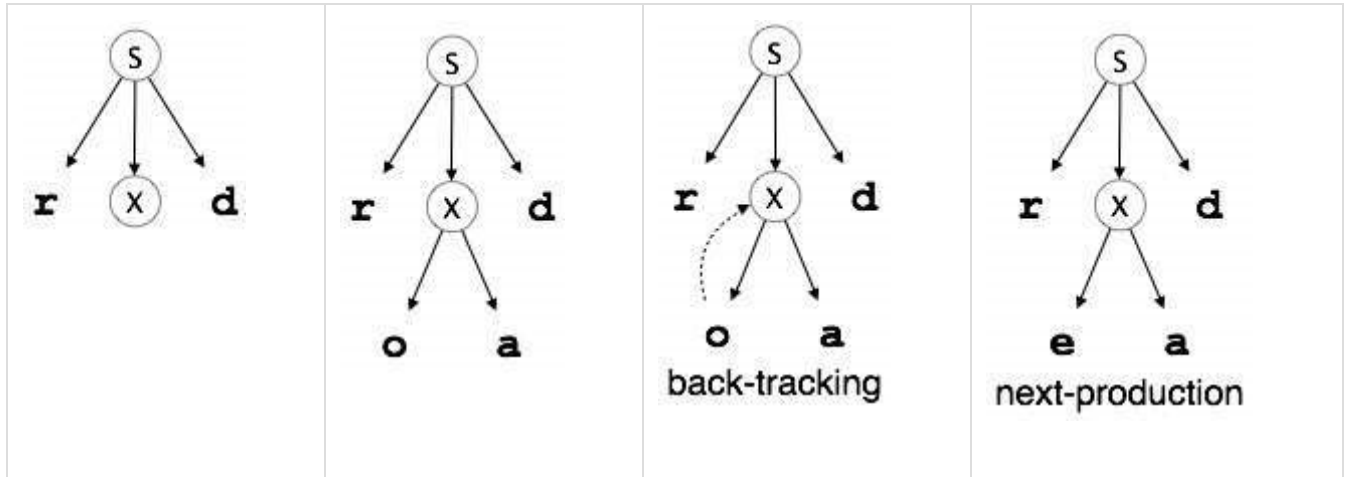
$$\begin{aligned} S &\rightarrow rXd \mid rZd \\ X &\rightarrow oa \mid ea \\ Z &\rightarrow ai \end{aligned}$$

For an input string: read, a top-down parser, will behave like this:

It will start with S from the production rules and will match its yield to the left-most letter of the input, i.e. 'r'. The very production of S ($S \rightarrow rXd$) matches with it. So the top-down parser

advances to the next input letter (i.e. 'e'). The parser tries to expand non-terminal 'X' and checks its production from the left ($X \rightarrow oa$). It does not match with the next input symbol. So the top-down parser backtracks to obtain the next production rule of X, ($X \rightarrow ea$).

Now the parser matches all the input letters in an ordered manner. The string is accepted.



2.2 Non Recursive Predictive Parser / LL (1) Parser

- Top Down parsing can be viewed as the process of constructing a parse tree for the input string, starting from the root and creating the nodes of the parse tree in preorder.
- Top-down parsing can also be viewed as finding a leftmost derivation for an input string.

Construction of Predictive LL (1) Parser

- Before constructing the Predictive LL (1) parsers we have to
 - Eliminate ambiguity,
 - Eliminate left-recursion and
 - Perform left factor where required.
- For construction of Predictive LL (1) parser, follow the following steps:
 - Computation of FIRST
 - Computation of FOLLOW
 - Construct the Predictive Parsing Table using FIRST and FOLLOW sets
- Parse the input string

Computation of FIRST: The Computation of FIRST allows the parser to choose which production to apply, based on the first input symbol.

Definition – FIRST: FIRST (α), where α is any string of grammar symbols, to be the set of terminals that begin strings derived from α .

Algorithm

- (R1) If x is terminal then **FIRST(x) = { x }**
- (R2) If there is a production $A \rightarrow \epsilon$, then **FIRST (A) = { ϵ }**
- (R3) If $A \rightarrow X_1X_2X_3 \dots X_k$ is a production, where X is a grammar symbol, then
- If **FIRST(X_1) $\neq \{\epsilon\}$** then **FIRST (A) = FIRST(X_1)**.
 - If **FIRST(X_1) = { ϵ }** then **FIRST (A) = FIRST(X_2) - { ϵ }**.
 - If **FIRST(X_1) = { ϵ }** and **FIRST (X_2) = { ϵ }** then **FIRST(A) = FIRST(X_3) - { ϵ }**.
 - ...
 - If **ϵ is in the FIRST set for every X_k** then **FIRST (A) = ϵ**

Computation of FOLLOW: Follow (A) for a non-terminal A, to be the set of terminals a such that a can appear immediately to the right of A in some sentential form.

Algorithm

- (R1) Include **$\$$ in the Follow (S)**, where S is the start symbol of the grammar
- (R2) If $A \rightarrow \alpha B \beta$, then **FOLLOW (B) = FIRST (β) except ϵ**
- (R3) If $(A \rightarrow \alpha B)$ or $(A \rightarrow \alpha B \beta \text{ and FIRST } (\beta) \text{ has } \epsilon)$, then **FOLLOW (B) = FOLLOW (A)**

Construction of Parsing Table – Algorithm

- (R1) For each terminal ' a ' in FIRST (α), add **$A \rightarrow \alpha$ to M [A, a]**
- (R2) If ϵ is in FIRST (α), then add **$A \rightarrow \epsilon$ to M [A, b]** for every symbol ' b ' in FOLLOW (A).
- (R3) If ϵ is in FIRST (α) and $\$$ is in FOLLOW (A) then Add **$A \rightarrow \alpha$ to M [A, $\$$]**.
- (R4) Make all undefined entries of M be **ERROR**.

Parsing the Input string - Algorithm

Let X is the symbol on top of the stack and ' a ' is the current input symbol,

- (R1) If X is a terminal and $X=a=S$, then the parser halts and announce the successful completion of parsing.
- (R2) If X is a terminal and $X=a \neq S$, then Pop-off X from the stack and advance the input pointer.
- (R3) If X is a Non-terminal then consult an entry $M [X, a]$ of the parsing table,
- If $M [X, a] = \{X \rightarrow UVW\}$, then the parser replaces X on top of the stack by UVW , with U on top of the stack.
 - If $M [X, a] = \text{Undefined}$, then the parser calls an error recovery routine.

Example:

Construct the LL (1) parsing table for the following grammar and parse the string $id + id * id$.

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid id$

Given:

G: $E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow (E) \mid id$

Step -1: Elimination of Left Recursion

After Eliminating the Left Recursion from E and T productions,

$E \rightarrow TE'$
 $E' \rightarrow +TE' \mid \epsilon$
 $T \rightarrow FT'$
 $T' \rightarrow *FT' \mid \epsilon$
 $F \rightarrow (E) \mid id$

Step -2: Computation of FIRST

FIRST (E)

$E \rightarrow TE'$

First (E) = First (T).....(1)

FIRST (E')

$E' \rightarrow +TE' \quad E' \rightarrow \epsilon$

First (E') = {+, ϵ }

FIRST (T)

$T \rightarrow FT'$

First (T) = First (F).....(2)

FIRST (T')

$T' \rightarrow *FT' \quad T' \rightarrow \epsilon$

First (T') = {*, ϵ }

FIRST (F)

$F \rightarrow (E) \quad F \rightarrow id$

First (F) = {(, id}

Substituting First (F) in (2)

First (T) = First (F) = {(, id}

Substituting First (T) in (1)

First (E) = First (T) = First (F) = {(, id}

Step -3: Computation of FOLLOW

FOLLOW (E)

FOLLOW (E) = {\$} by [R1]

$F \rightarrow (E)$

FOLLOW (E) = First (E) = {\$,)} by [R2]

FOLLOW (E')

$E \rightarrow TE'$

Follow(E') = Follow(E) by [R3] = {\$,)}

$E' \rightarrow +TE'$

Follow(E') = Follow(E') by [R3] = {\$,)}

FOLLOW (T)

$E \rightarrow TE'$

Follow(T) = First (E') Except ϵ by (R2) = {+}

Follow(T) = Follow (E) Since First(E') has ϵ

Follow(T) = {+, \$,)}

$E' \rightarrow +TE'$

Follow (T) = Follow (E') by [R3] = {+, \$,)}

FOLLOW (T')

$T \rightarrow FT'$

Follow(T') = Follow (T) by [R3] = {+, \$,)}

$T' \rightarrow *FT'$

Follow(T') = Follow(T') by [R3] = {+, \$,) }

FOLLOW (F)

T → FT'

Follow(F) = First (T') Except ε by (R2) = { * }

Follow(F) = Follow (T) Since First(E') has ε

Follow (F) = Follow(T) = {+, \$,) }

T' → *FT'

Follow (F) = Follow (T') by [R3] = { *, +, \$,) }

Symbol	FIRST	FOLLOW
E	(, id	\$,)
E'	+, ε	\$,)
T	(, id	+, \$,)
T'	*, ε	+, \$,)
F	(, id	*, +, \$,)

Step – 4: Construction of Parsing Table

M	id	+	*	(	)	\$
E	E → TE'			E → TE'		
E'		E' → +TE'			E' → ε	E' → ε
T	T → FT'			T → FT'		
T'		T' → ε	T' → *FT'		T' → ε	T' → ε
F	F → id				F → (E)	

Step 5: Parsing the String id+id*id

Stack	Input buffer	Action taken
\$E	id+id*id\$	M[E,id] = E → TE'
\$E'T	id+id*id\$	M[T,id] = T → FT'
\$E'T'F	id+id*id\$	M[F,id] = F → id
\$E'T' id	id +id*id\$	Pop off & Advance the Input pointer
\$E'T'	+id*id\$	M[T',+] = T' → ε
\$E'	+id*id\$	M[E',+] = E' → +TE'
\$E'T+	+id*id\$	Pop off & Advance the Input pointer
\$E'T	id*id\$	M[T,id] = T → FT'
\$E'T'F	id*id\$	M[F,id] = F → id
\$E'T' id	id *id\$	Pop off & Advance the Input pointer

\$E'T'	*id\$	$M[T', *] = T' \rightarrow *FT'$
\$E'T'F*	*id\$	Pop off & Advance the Input pointer
\$E'T'F	id\$	$M[F, id] = F \rightarrow id$
\$E'T'id	id\$	Pop off & Advance the Input pointer
\$E'T'	\$	$M[T', \$] = T' \rightarrow \epsilon$
\$E'	\$	$M[E', \$] = E' \rightarrow \epsilon$
\$	\$	Accepted

More problems on LL (1) Parser: please refer the class work note-book.

IV. YACC – PARSER GENERATOR

A parser generator is a program that takes as input a specification of a syntax and produces as output a procedure for recognizing that language. Historically, they are also called compiler-compilers.

YACC (yet another compiler-compiler) is an LALR(1) parser generator. YACC was originally designed for being complemented by Lex.

Input File:

YACC input file is divided in three parts.

```

/* definitions */
....
%%
/* rules */
....
%%
/* auxiliary routines */
....

```

Input File: Definition Part:

- The definition part includes information about the tokens used in the syntax definition:
- %token NUMBER
- YACC automatically assigns numbers for tokens. YACC also recognizes single characters as tokens. Therefore, assigned token numbers should no overlap ASCII codes.
- The definition part can include C code external to the definition of the parser and variable declarations, within %{ and %} in the first column.

- It can also include the specification of the starting symbol in the grammar:
%start nonterminal

Input File: Rule Part:

- The rules part contains grammar definition in a modified BNF form.
- Actions is C code in { } and can be embedded inside (Translation schemes).

Input File: Auxiliary Routines Part:

- The auxiliary routines part is only C code.
- It includes function definitions for every function needed in rules part.
- It can also contain the main() function definition if the parser is going to be run as a program.
- The main() function must call the function yyparse().

Input File:

- If yylex() is not defined in the auxiliary routines sections, then it should be included:
#include "lex.yy.c"
- YACC input file generally finishes with: * .y

Output Files:

- The output of YACC is a file named **y.tab.c**
- If it contains the **main()** definition, it must be compiled to be executable.
- Otherwise, the code can be an external function definition for the function **int yyparse()**
- If called with the **-d** option in the command line, Yacc produces as output a header file **y.tab.h** with all its specific definition (particularly important are token definitions to be included, for example, in a Lex input file).
- If called with the **-v** option, Yacc produces as output a file **y.output** containing a textual description of the LALR(1) parsing table used by the parser. This is useful for tracking down how the parser solves conflicts.